

```

class PDFMCQGenerator {
  constructor() {
    this.pdfDoc = null;
    this.currentPage = 1;
    this.mcqs = [];
    this.currentQuestion = 0;
    this.userAnswers = {};
    this.score = 0;

    this.init();
  }

  init() {
    this.bindEvents();
    pdfjsLib.GlobalWorkerOptions.workerSrc =
'https://cdnjs.cloudflare.com/ajax/libs/pdf.js/3.11.174/pdf.worker.min.js';
  }

  bindEvents() {
    const uploadBtn = document.getElementById('uploadBtn');
    const pdfInput = document.getElementById('pdfInput');
    const uploadArea = document.getElementById('uploadArea');
    const generateMCQs = document.getElementById('generateMCQs');
    const downloadMCQs = document.getElementById('downloadMCQs');

    uploadBtn.addEventListener('click', () => pdfInput.click());
    uploadArea.addEventListener('click', () => pdfInput.click());

    pdfInput.addEventListener('change', (e) => this.handlePDFUpload(e));
    generateMCQs.addEventListener('click', () => this.generateMCQs());
    downloadMCQs.addEventListener('click', () => this.downloadMCQs());

    // Quiz navigation
    document.getElementById('nextBtn').addEventListener('click', () =>
this.nextQuestion());
    document.getElementById('prevBtn').addEventListener('click', () =>
this.prevQuestion());
    document.getElementById('submitQuiz').addEventListener('click', () =>
this.submitQuiz());
    document.getElementById('restartQuiz').addEventListener('click', () =>
this.restartQuiz());
  }

  async handlePDFUpload(event) {
    const file = event.target.files[0];
    if (!file) return;

    const arrayBuffer = await file.arrayBuffer();

```

```

    this.pdfDoc = await pdfjsLib.getDocument(arrayBuffer).promise;

    document.getElementById('uploadArea').style.display = 'none';
    document.getElementById('pdfPreview').style.display = 'block';

    this.renderPage(1);
  }

  async renderPage(num) {
    const page = await this.pdfDoc.getPage(num);
    const viewport = page.getViewport({ scale: 1.5 });
    const canvas = document.getElementById('pdfCanvas');
    const context = canvas.getContext('2d');

    canvas.height = viewport.height;
    canvas.width = viewport.width;

    await page.render({
      canvasContext: context,
      viewport: viewport
    }).promise;
  }

  generateMCQs() {
    // Simulate MCQ generation from PDF content
    // In a real app, you'd use OCR/NLP APIs like Google Vision + GPT
    this.mcqs = this.createSampleMCQs(20); // Generate 20 MCQs

    document.getElementById('pdfPreview').style.display = 'none';
    document.getElementById('quizSection').style.display = 'block';
    document.getElementById('downloadMCQs').style.display = 'inline-block';

    this.showQuestion(0);
  }

  createSampleMCQs(count) {
    const subjects = ['Maths', 'Reasoning', 'English', 'GK', 'Science'];
    const topics = ['Algebra', 'Geometry', 'Percentage', 'Analogy', 'Grammar', 'History',
'Physics'];
    const mcqs = [];

    for (let i = 0; i < count; i++) {
      const subject = subjects[Math.floor(Math.random() * subjects.length)];
      const topic = topics[Math.floor(Math.random() * topics.length)];

      mcqs.push({
        id: i,
        question: this.generateQuestion(subject, topic),

```

```

        options: this.generateOptions(subject, topic),
        correct: Math.floor(Math.random() * 4),
        explanation: this.generateExplanation(subject, topic),
        subject: subject,
        topic: topic
    });
}
return mcqs;
}

generateQuestion(subject, topic) {
    const questions = {
        'Maths': `If  $x + y = 15$  and  $x - y = 5$ , then what is the value of  $x^2 - y^2$ ? (${topic})`,
        'Reasoning': `Find the next number in series: 2, 6, 12, 20, ? (${topic})`,
        'English': `Choose the correct synonym of "Eloquent": (${topic})`,
        'GK': `Who was the first President of India? (${topic})`,
        'Science': `What is the chemical formula of water? (${topic})`
    };
    return questions[subject] || `Sample ${subject} question on ${topic}`;
}

generateOptions(subject, topic) {
    const options = ['Option A', 'Option B', 'Option C', 'Option D'];
    return options.map(opt => opt);
}

generateExplanation(subject, topic) {
    const explanations = {
        'Maths': ` $x^2 - y^2 = (x+y)(x-y) = 15 \times 5 = 75$ `,
        'Reasoning': `Pattern: +4, +6, +8, +10 → Answer: 30`,
        'English': `"Eloquent" means fluent or persuasive in speech`,
        'GK': `Dr. Rajendra Prasad was the first President of India`,
        'Science': `H2O - two hydrogen atoms and one oxygen atom`
    };
    return explanations[subject] || `Detailed explanation for ${subject} - ${topic}`;
}

showQuestion(index) {
    const mcq = this.mcqs[index];
    this.currentQuestion = index;

    document.getElementById('questionNum').textContent = index + 1;
    document.getElementById('totalQuestions').textContent = this.mcqs.length;
    document.getElementById('currentQuestion').innerHTML = `
        <strong>Q${index + 1}</strong> ${mcq.question}
        <div style="font-size: 14px; color: #666; margin-top: 10px;">
            <strong>Subject:</strong> ${mcq.subject} | <strong>Topic:</strong> ${mcq.topic}
        </div>
    `;
}

```

```

`;

const optionsContainer = document.getElementById('options');
optionsContainer.innerHTML = "";

mcq.options.forEach((option, optIndex) => {
  const optionEl = document.createElement('div');
  optionEl.className = 'option';
  optionEl.innerHTML = `
    <strong>${String.fromCharCode(65 + optIndex)}.</strong> ${option}
  `;
  optionEl.addEventListener('click', () => this.selectOption(index, optIndex));
  optionsContainer.appendChild(optionEl);
});

// Show previous answer if exists
if (this.userAnswers[index] !== undefined) {
  const selectedOption =
document.querySelectorAll('.option')[this.userAnswers[index]];
  selectedOption.classList.add('selected');
}

// Update navigation buttons
document.getElementById('prevBtn').style.display = index > 0 ? 'inline-block' : 'none';
document.getElementById('nextBtn').style.display = index < this.mcqs.length - 1 ?
'inline-block' : 'none';
document.getElementById('submitQuiz').style.display = index === this.mcqs.length - 1
? 'inline-block' : 'none';
}

selectOption(questionIndex, optionIndex) {
  this.userAnswers[questionIndex] = optionIndex;

  // Visual feedback
  const options = document.querySelectorAll('.option');
  options.forEach((opt, idx) => {
    opt.classList.remove('selected', 'correct', 'incorrect');
    if (idx === this.mcqs[questionIndex].correct) {
      opt.classList.add('correct');
    }
    if (idx === optionIndex) {
      opt.classList.add('selected');
      if (idx !== this.mcqs[questionIndex].correct) {
        opt.classList.add('incorrect');
      }
    }
  });
});

```

```

// Show explanation
setTimeout(() => {
  const feedback = document.createElement('div');
  feedback.className = `feedback ${optionIndex ===
this.mcqs[questionIndex].correct ? 'correct' : 'incorrect'}`;
  feedback.innerHTML = `
    <strong>${optionIndex === this.mcqs[questionIndex].correct ? '✅ Correct!' : '❌
Incorrect'}</strong><br>
    <strong>Right Answer:</strong> ${String.fromCharCode(65 +
this.mcqs[questionIndex].correct)}<br>
    <strong>Explanation:</strong> ${this.mcqs[questionIndex].explanation}
  `;

  const container = document.querySelector('.question-container');
  const existingFeedback = container.querySelector('.feedback');
  if (existingFeedback) existingFeedback.remove();
  container.appendChild(feedback);
}, 300);
}

nextQuestion() {
  if (this.currentQuestion < this.mcqs.length - 1) {
    this.showQuestion(this.currentQuestion + 1);
  }
}

prevQuestion() {
  if (this.currentQuestion > 0) {
    this.showQuestion(this.currentQuestion - 1);
  }
}

submitQuiz() {
  this.score = Object.values(this.userAnswers).filter((ans, idx) =>
    ans === this.mcqs[idx].correct
  ).length;

  document.getElementById('quizSection').style.display = 'none';
  document.getElementById('results').style.display = 'block';
  document.getElementById('finalScore').textContent = `${this.score}/${this.mcqs.length}
(${Math.round((this.score/this.mcqs.length)*100)}%)`;
}

restartQuiz() {
  this.currentQuestion = 0;
  this.userAnswers = {};
  this.score = 0;
  document.getElementById('quizSection').style.display = 'block';
}

```

```

        document.getElementById('results').style.display = 'none';
        this.showQuestion(0);
    }

    downloadMCQs() {
        const quizData = this.mcqs.map((mcq, idx) => `
Q${idx+1}. ${mcq.question}
A) ${mcq.options[0]}   B) ${mcq.options[1]}
C) ${mcq.options[2]}   D) ${mcq.options[3]}
Correct: ${String.fromCharCode(65 + mcq.correct)}
Explanation: ${mcq.explanation}

`.join('\n');

        const blob = new Blob([quizData], { type: 'text/plain' });
        const url = URL.createObjectURL(blob);
        const a = document.createElement('a');
        a.href = url;
        a.download = 'SSC_Mock_Test_Questions.txt';
        a.click();
    }
}

// Initialize app when page loads
document.addEventListener('DOMContentLoaded', () => {
    new PDFMCQGenerator();
});

```